

INSTITUTO INFNET
ESCOLA SUPERIOR DE TECNOLOGIA
GRADUAÇÃO EM ENGENHARIA DE SOFTWARE



Fundamentos de Modelagem Relacional e SQL

TP 3

Alunos: Richard de Jesus Cabral Alves.

9 de nov. de 2025

Clínica Veterinária Aurora

A Clínica Veterinária Aurora precisa de um pequeno banco de dados relacional para registrar tutores, veterinários, pets e consultas.

Com base no texto abaixo, crie todas as tabelas necessárias, com os nomes, colunas, tamanhos e constraints exatamente conforme especificado.

Requisitos do Modelo Lógico

1. Tabela tutores

- Armazena os donos dos animais. Cada tutor deve possuir:
 - um identificador único;
 - nome;
 - telefone;
 - email;
 - data de criação do registro.
- Regras:
 - A tabela deve se chamar exatamente tutores.
 - O campo id deve ser chave primária e autoincrementável.
 - nome é obrigatório.
 - Os tamanhos dos campos devem ser:
 - nome → VARCHAR(150)
 - telefone → VARCHAR(30)
 - email → VARCHAR(150)
 - criado_em deve ter DEFAULT NOW().
-

2. Tabela veterinarios

- Armazena os profissionais da clínica.
- Campos obrigatórios: id, nome, CRMV, especialidade, status ativo, criado_em.
- Regras:

- Nome da tabela deve ser exatamente veterinarios.
- id é chave primária autoincrementável.
- nome é obrigatório (VARCHAR(150)).
- crmv deve ter UNIQUE (VARCHAR(50)).
- especialidade → VARCHAR(100).
- ativo deve ser BOOLEAN com DEFAULT TRUE.
- criado_em deve ter DEFAULT NOW().

○

3. Tabela pets

- Armazena os animais dos tutores.
- Campos necessários:
 - id;
 - nome;
 - especie;
 - data_nascimento;
 - tutor_id;
 - criado_em.
- Regras:
 - Nome deve ser exatamente pets.
 - id é chave primária autoincrementável.
 - nome → VARCHAR(100) NOT NULL.
 - especie → VARCHAR(50) NOT NULL.
 - data_nascimento → DATE.
 - tutor_id é obrigatório e referencia tutores(id).
 - criado_em → TIMESTAMP DEFAULT NOW().
 - Deve existir uma constraint de chave estrangeira com nome fk_pets_tutor.

○

4. Tabela consultas

- Registra os atendimentos realizados.
- Campos necessários:
 - id;
 - pet_id;
 - veterinario_id;
 - data_consulta;
 - motivo;
 - valor_consulta;
 - retorno_agendado_em;
 - criado_em.
- Regras:
 - Tabela deve ser exatamente consultas.
 - id é chave primária autoincrementável.
 - pet_id e veterinario_id são obrigatórios e devem referenciar suas tabelas.

- data_consulta é obrigatória.
- Constraint fk_consultas_pet deve referenciar pets(id).
- Constraint fk_consultas_veterinario deve referenciar veterinarios(id).
- motivo → VARCHAR(255).
- valor_consulta → NUMERIC(10,2).
- retorno_agendado_em pode ser NULL.
- criado_em → DEFAULT NOW().

Compatibilidade com o arquivo TXT

Um arquivo TXT anexo contém 20 linhas de INSERT INTO pets (...).

Sua modelagem deve ser totalmente compatível com este arquivo.

- Não altere nomes das colunas.
- Não altere tipos de dados.
- Não remova constraints obrigatórias.
- Não inclua colunas adicionais.

queries:

create table tutores (

id integer generated always as identity,

nome varchar(150) not null,

telefone varchar(30),

email varchar(150),

criado_em date default now()

);

```
create table veterinarios (  
    id integer generated always as identity,  
    nome varchar(150) not null,  
    CRMV varchar(50) unique,  
    especialidade varchar(100),  
    ativo bool default True,  
    criado_em date default now()  
);
```

```
create table pets (  
    id integer generated always as identity,  
    nome varchar(150) not null,  
    especie varchar(50) not null,  
    data_nascimento date,  
    tutor_id integer,  
    criado_em date default now()  
);
```

```
alter table pets add constraint fk_pets_tutor foreign key (tutor_id)  
REFERENCES tutores (id);
```

```
create table consultas (  
    id integer generated always as identity,  
    pet_id integer not null,  
    veterinario_id integer not null,  
    data_consulta date not null,  
    motivo varchar(255),  
    valor_consulta numeric(10,2),  
    retorno_agendado_em date,  
    criado_em date default now()  
);  
  
alter table consultas add constraint fk_consultas_pet foreign key (pet_id)  
REFERENCES pets (id);  
  
alter table consultas add constraint fk_consultas_veterinario foreign key  
(veterinario_id) REFERENCES veterinarios (id);
```

Questão 2 Inserção de Dados e Verificação com COUNT

A seguir, você encontrará quatro blocos de comandos INSERT, correspondentes às tabelas:

- tutores (20 registros)
- veterinarios (20 registros)
- pets (20 registros)
- consultas (60 registros)

Tarefa:

1. Execute todos os INSERTs exatamente como fornecidos no enunciado ou arquivo TXT anexado.

Não altere nomes de colunas, quantidades ou valores.
2. Após inserir todos os dados, escreva uma query para verificar a quantidade de registros em cada tabela.

A saída da sua query final deve retornar exatamente:

tabela	registro
tutores	20
veterinarios	20
pets	20
consultas	60

Você deve escrever a query que produz *EXATAMENTE* essa contagem. A correção considerará se sua consulta confirma corretamente as quantidades esperadas.

```
INSERT INTO tutores (id, nome, telefone, email, criado em) VALUES
(1, 'Marina Alves',      '(21) 98888-0001', 'marina.alves@example.com',      NOW()),
(2, 'Jorge Matos',      '(21) 98888-0002', 'jorge.matos@example.com',      NOW()),
(3, 'Bruna Freitas',    '(21) 98888-0003', 'bruna.freitas@example.com',    NOW()),
(4, 'Cecília Prado',    '(21) 98888-0004', 'cecilia.prado@example.com',    NOW()),
(5, 'Rafael Araújo',    '(21) 98888-0005', 'rafael.araujo@example.com',    NOW()),
```

```
(6, 'Fabiana Lopes', ' (21) 98888-0006', 'fabiana.lopes@example.com', NOW()),
(7, 'Carlos Henrique', ' (21) 98888-0007', 'carlos.henrique@example.com', NOW()),
(8, 'Paula Siqueira', ' (21) 98888-0008', 'paula.siqueira@example.com', NOW()),
(9, 'Felipe Moreira', ' (21) 98888-0009', 'felipe.moreira@example.com', NOW()),
(10, 'Juliana Bueno', ' (21) 98888-0010', 'juliana.bueno@example.com', NOW()),
(11, 'Eduardo Lima', ' (21) 98888-0011', 'eduardo.lima@example.com', NOW()),
(12, 'Camila Santos', ' (21) 98888-0012', 'camila.santos@example.com', NOW()),
(13, 'Marcos Vinícius', ' (21) 98888-0013', 'marcos.vinicius@example.com', NOW()),
(14, 'Larissa Ribeiro', ' (21) 98888-0014', 'larissa.ribeiro@example.com', NOW()),
(15, 'Renata Silva', ' (21) 98888-0015', 'renata.silva@example.com', NOW()),
(16, 'Tiago Andrade', ' (21) 98888-0016', 'tiago.andrade@example.com', NOW()),
(17, 'Sandra Moreira', ' (21) 98888-0017', 'sandra.moreira@example.com', NOW()),
(18, 'Henrique Souza', ' (21) 98888-0018', 'henrique.souza@example.com', NOW()),
(19, 'Marta Cardoso', ' (21) 98888-0019', 'marta.cardoso@example.com', NOW()),
(20, 'Bruno Rezende', ' (21) 98888-0020', 'bruno.rezende@example.com', NOW());
```

```
INSERT INTO veterinarios (id, nome, crmv, especialidade, ativo, criado_em) VALUES
(1, 'Ana Paula Nogueira', 'CRMV-RJ-1001', 'Clínico Geral', TRUE, NOW()),
(2, 'Bruno Carvalho', 'CRMV-RJ-1002', 'Cirurgia Geral', TRUE, NOW()),
(3, 'Carla Menezes', 'CRMV-RJ-1003', 'Dermatologia', TRUE, NOW()),
(4, 'Daniel Farias', 'CRMV-RJ-1004', 'Ortopedia', TRUE, NOW()),
(5, 'Elaine Souza', 'CRMV-RJ-1005', 'Oftalmologia', TRUE, NOW()),
(6, 'Fernando Rocha', 'CRMV-RJ-1006', 'Emergência', TRUE, NOW()),
(7, 'Gabriela Torres', 'CRMV-RJ-1007', 'Oncologia', TRUE, NOW()),
(8, 'Henrique Teixeira', 'CRMV-RJ-1008', 'Anestesiologia', TRUE, NOW()),
(9, 'Isabela Martins', 'CRMV-RJ-1009', 'Gatos (Felinos)', TRUE, NOW()),
(10, 'João Victor Lima', 'CRMV-RJ-1010', 'Cães (Caninos)', TRUE, NOW()),
(11, 'Karina Duarte', 'CRMV-RJ-1011', 'Animais Exóticos', TRUE, NOW()),
(12, 'Lucas Almeida', 'CRMV-RJ-1012', 'Clínico Geral', TRUE, NOW()),
(13, 'Mariana Gomes', 'CRMV-RJ-1013', 'Cardiologia', TRUE, NOW()),
(14, 'Nelson Ribeiro', 'CRMV-RJ-1014', 'Neurologia', TRUE, NOW()),
(15, 'Olívia Fernandes', 'CRMV-RJ-1015', 'Endocrinologia', TRUE, NOW()),
(16, 'Pedro Henrique Dias', 'CRMV-RJ-1016', 'Clínico Geral', TRUE, NOW()),
(17, 'Quitéria Barros', 'CRMV-RJ-1017', 'Dermatologia', TRUE, NOW()),
(18, 'Ricardo Monteiro', 'CRMV-RJ-1018', 'Cirurgia Geral', TRUE, NOW()),
(19, 'Simone Freitas', 'CRMV-RJ-1019', 'Gatos (Felinos)', TRUE, NOW()),
(20, 'Thiago Costa', 'CRMV-RJ-1020', 'Animais Silvestres', TRUE, NOW());
```

```
INSERT INTO pets (id, nome, especie, data_nascimento, tutor_id, criado_em) VALUES
(1, 'Thor', 'cachorro', '2018-05-11', 1, NOW()),
(2, 'Mima', 'gato', '2020-08-22', 2, NOW()),
(3, 'Algodão', 'coelho', '2021-02-10', 3, NOW()),
(4, 'Luna', 'cachorro', '2019-12-01', 4, NOW()),
(5, 'Nino', 'gato', '2017-04-19', 5, NOW()),
(6, 'Pipoca', 'hamster', '2022-01-17', 6, NOW()),
(7, 'Bento', 'cachorro', '2016-09-30', 7, NOW()),
(8, 'Mel', 'gato', '2023-03-05', 8, NOW()),
(9, 'Kiwi', 'tartaruga', '2015-11-12', 9, NOW()),
(10, 'Zara', 'cachorro', '2020-07-14', 10, NOW()),
(11, 'Tico', 'pássaro', '2022-10-03', 11, NOW()),
(12, 'Amora', 'gato', '2019-01-28', 12, NOW()),
(13, 'Frodo', 'cachorro', '2018-06-16', 13, NOW()),
(14, 'Malu', 'gato', '2021-11-09', 14, NOW()),
(15, 'Bolt', 'cachorro', '2017-02-23', 15, NOW()),
(16, 'Sakura', 'coelho', '2020-10-15', 16, NOW()),
(17, 'Loki', 'gato', '2022-05-07', 17, NOW()),
(18, 'Koda', 'cachorro', '2021-04-11', 18, NOW()),
(19, 'Jade', 'gato', '2016-08-08', 19, NOW()),
(20, 'Snow', 'coelho', '2019-09-21', 20, NOW());
```

```
INSERT INTO consultas (id, pet_id, veterinario_id, data_consulta, motivo,
valor_consulta, retorno_agendado_em, criado_em) VALUES
(21, 1, 1, '2024-08-05 10:00', 'Vacinação anual', 130.00, NULL, NOW());
```

```

(22, 2, 9, '2024-08-07 11:15', 'Retorno alergia', 150.00, NULL, NOW()),
(23, 3, 11, '2024-08-10 09:30', 'Vermifugação', 110.00, NULL, NOW()),
(24, 4, 4, '2024-08-12 16:00', 'Retorno dor na pata', 180.00, NULL, NOW()),
(25, 5, 19, '2024-08-15 14:10', 'Check-up felino', 160.00, NULL, NOW()),
(26, 6, 6, '2024-08-18 15:40', 'Crise respiratória', 260.00, '2024-09-01 15:40',
NOW()),
(27, 7, 10, '2024-08-20 13:20', 'Tosse leve', 140.00, NULL, NOW()),
(28, 8, 9, '2024-08-22 10:05', 'Dermatite recorrente', 190.00, '2024-09-22 10:05',
NOW()),
(29, 9, 20, '2024-08-25 09:10', 'Revisão casco', 130.00, NULL, NOW()),
(30, 10, 1, '2024-08-28 14:30', 'Vacinação múltipla', 140.00, NULL, NOW()),

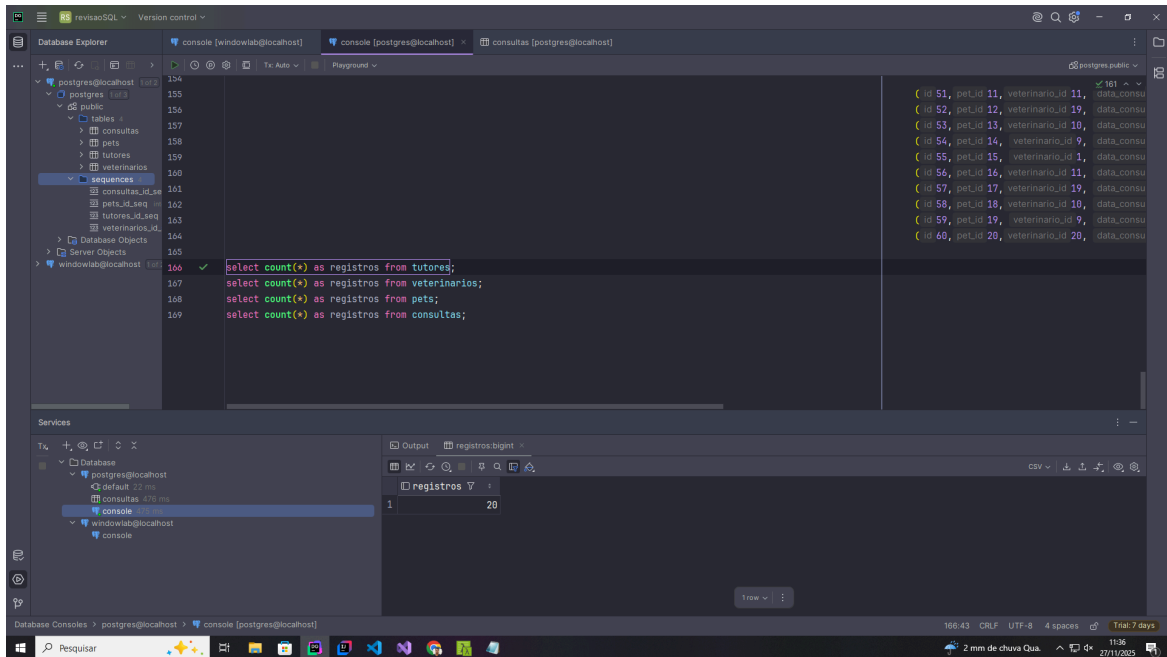
(31, 11, 11, '2024-09-02 15:00', 'Queda de penas', 135.00, NULL, NOW()),
(32, 12, 19, '2024-09-05 09:20', 'Infecção ocular', 195.00, '2024-09-19 09:20',
NOW()),
(33, 13, 10, '2024-09-07 11:10', 'Dor abdominal', 180.00, NULL, NOW()),
(34, 14, 9, '2024-09-10 16:15', 'Alergia alimentar', 190.00, NULL, NOW()),
(35, 15, 1, '2024-09-12 14:00', 'Check-up geriátrico', 150.00, NULL, NOW()),
(36, 16, 11, '2024-09-15 13:40', 'Letargia persistente', 170.00, NULL, NOW()),
(37, 17, 19, '2024-09-18 09:05', 'Gastrite felina', 205.00, NULL, NOW()),
(38, 18, 10, '2024-09-20 10:25', 'Otite canina', 155.00, '2024-10-04 10:25',
NOW()),
(39, 19, 9, '2024-09-23 15:10', 'Dermatite crônica', 200.00, NULL, NOW()),
(40, 20, 20, '2024-09-27 08:50', 'Apatia e perda de apetite', 175.00, NULL, NOW()),

(41, 1, 3, '2024-10-01 10:00', 'Coceira nas orelhas', 185.00, NULL, NOW()),
(42, 2, 9, '2024-10-03 11:30', 'Dermatite felina', 195.00, '2024-10-24 11:30',
NOW()),
(43, 3, 11, '2024-10-05 09:15', 'Check-up coelho', 120.00, NULL, NOW()),
(44, 4, 4, '2024-10-07 16:00', 'Ortopedia acompanhamento', 220.00, NULL, NOW()),
(45, 5, 19, '2024-10-10 14:05', 'Consulta de rotina', 140.00, NULL, NOW()),
(46, 6, 6, '2024-10-13 15:30', 'Emergência respiratória', 300.00, '2024-10-20 15:30',
NOW()),
(47, 7, 10, '2024-10-15 13:20', 'Check-up canino', 135.00, NULL, NOW()),
(48, 8, 9, '2024-10-18 10:00', 'Controle de alergia', 175.00, NULL, NOW()),
(49, 9, 20, '2024-10-20 09:25', 'Exame em tartaruga', 160.00, NULL, NOW()),
(50, 10, 1, '2024-10-23 14:45', 'Vacinação reforço', 145.00, NULL, NOW()),

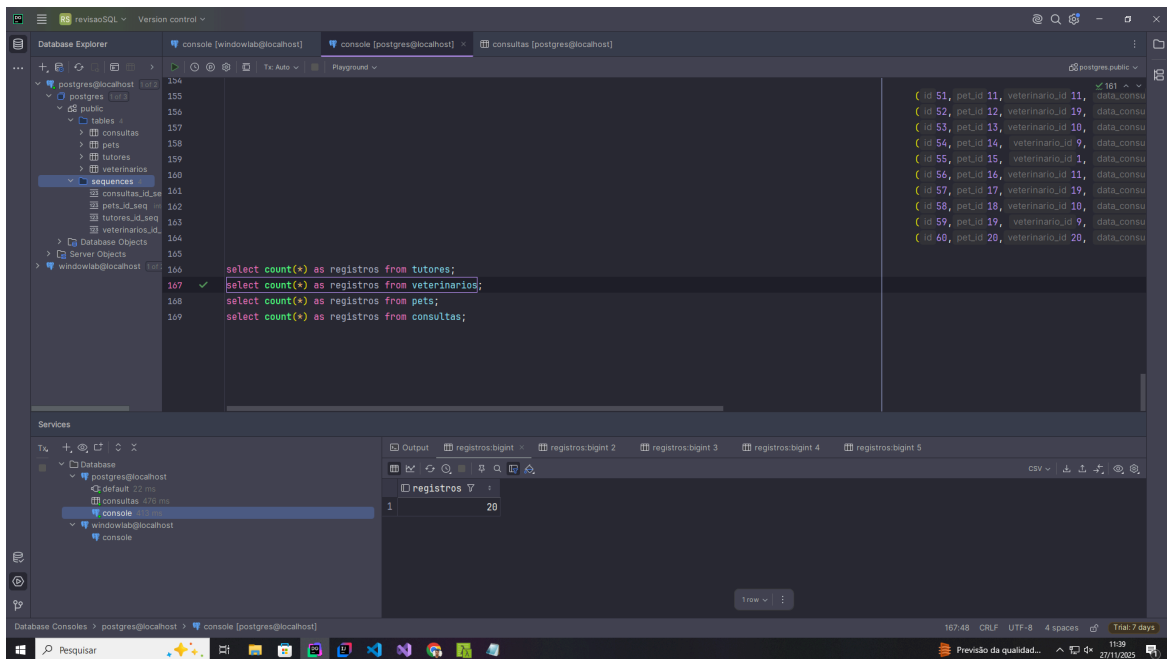
(51, 11, 11, '2024-11-02 15:10', 'Revisão penas', 140.00, NULL, NOW()),
(52, 12, 19, '2024-11-05 09:00', 'Retorno infecção ocular', 180.00, NULL, NOW()),
(53, 13, 10, '2024-11-07 11:35', 'Diarreia aguda', 170.00, NULL, NOW()),
(54, 14, 9, '2024-11-10 16:20', 'Controle alergia alimentar', 200.00, NULL, NOW()),
(55, 15, 1, '2024-11-12 14:15', 'Check-up anual', 130.00, NULL, NOW()),
(56, 16, 11, '2024-11-15 13:30', 'Avaliação geral', 150.00, NULL, NOW()),
(57, 17, 19, '2024-11-18 09:40', 'Gastrite leve', 185.00, NULL, NOW()),
(58, 18, 10, '2024-11-20 10:10', 'Otite recorrente', 165.00, '2024-12-04 10:10',
NOW()),
(59, 19, 9, '2024-11-23 15:05', 'Dermatite em remissão', 175.00, NULL, NOW()),
(60, 20, 20, '2024-11-27 08:55', 'Retorno apatia', 160.00, NULL, NOW());

```

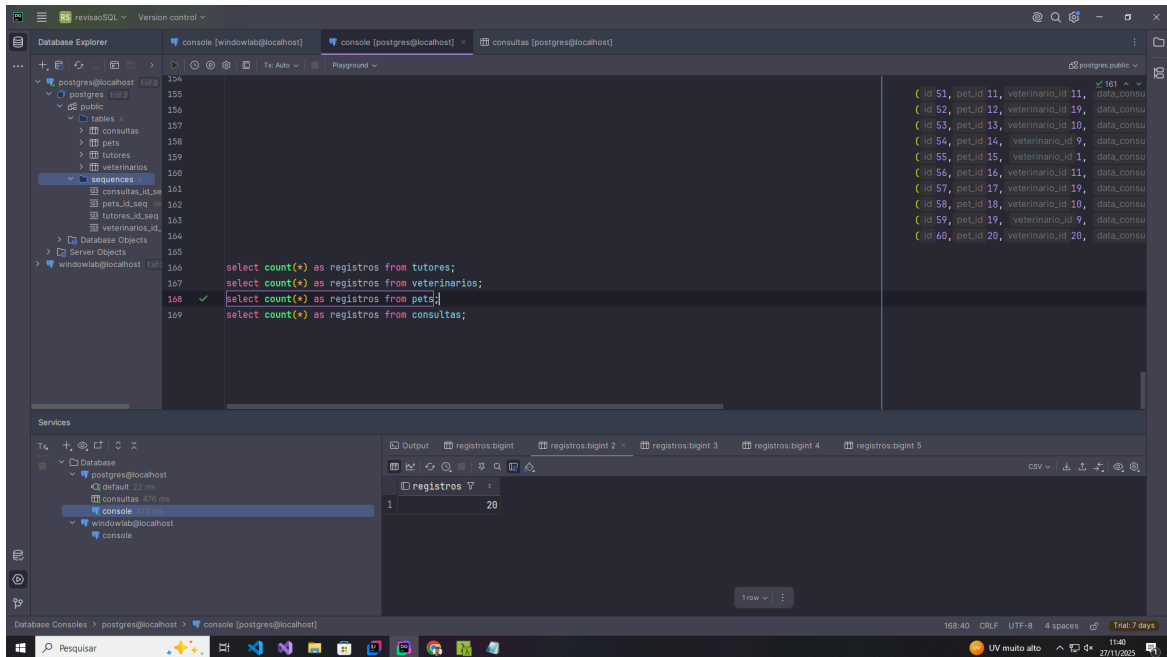
Tutores:



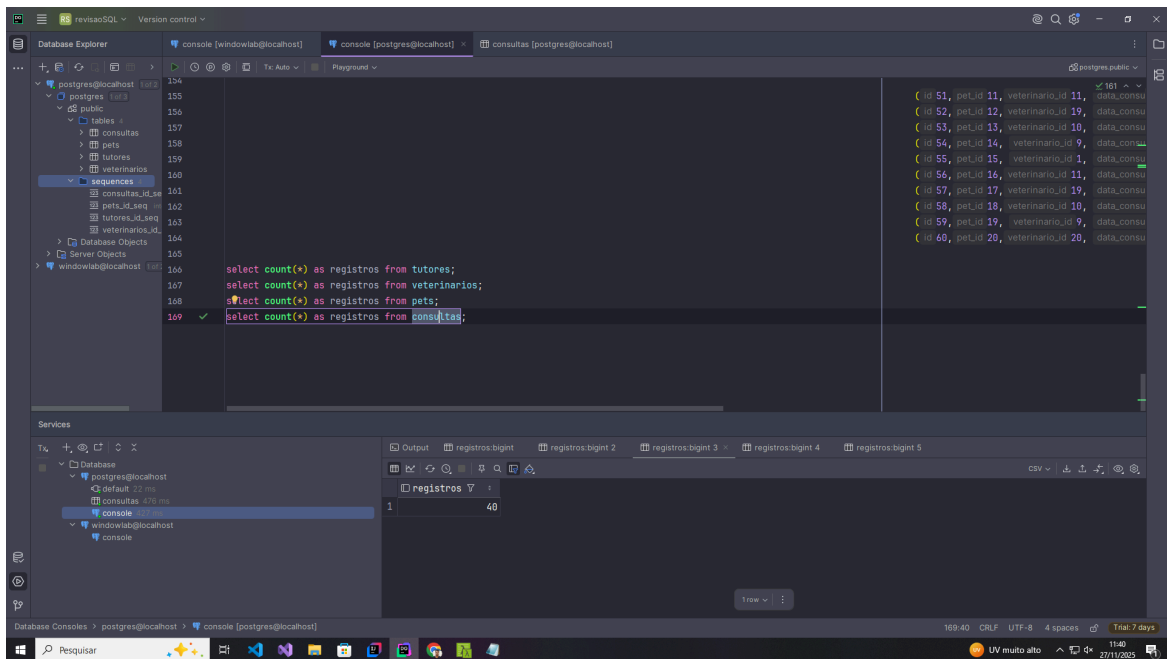
Veterinarios:



Pets:



Consultas:

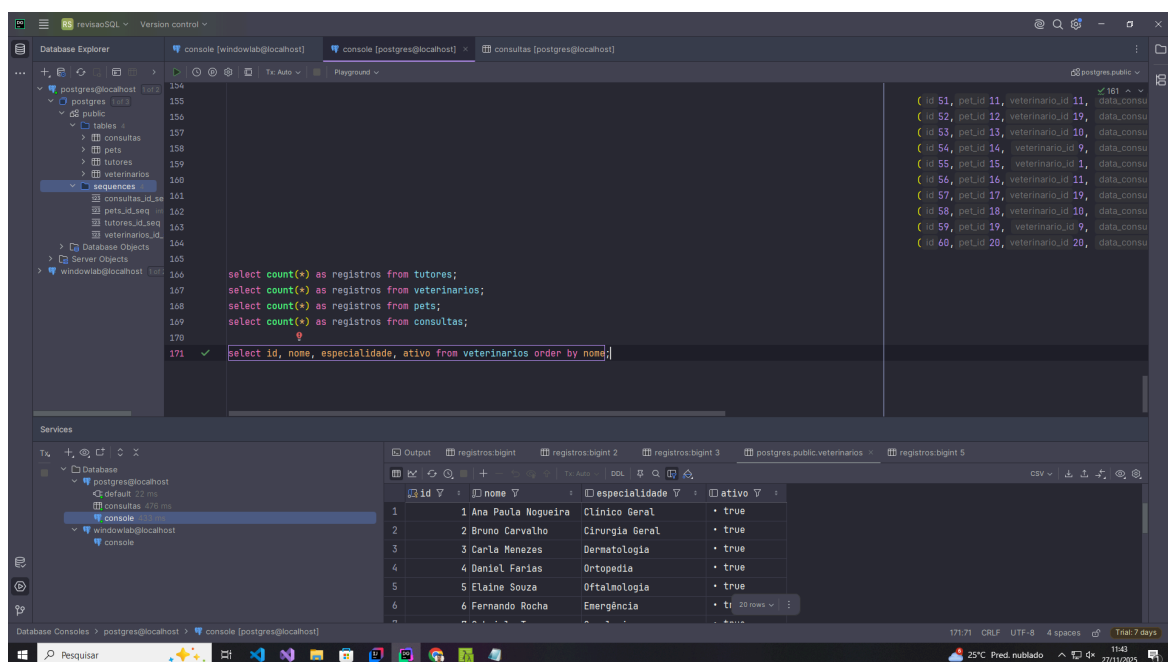


Questão 3 Listagem básica de veterinários

Liste todos os veterinários cadastrados, exibindo as colunas:

- id
- nome
- especialidade
- ativo

Ordene o resultado pelo nome em ordem alfabética.



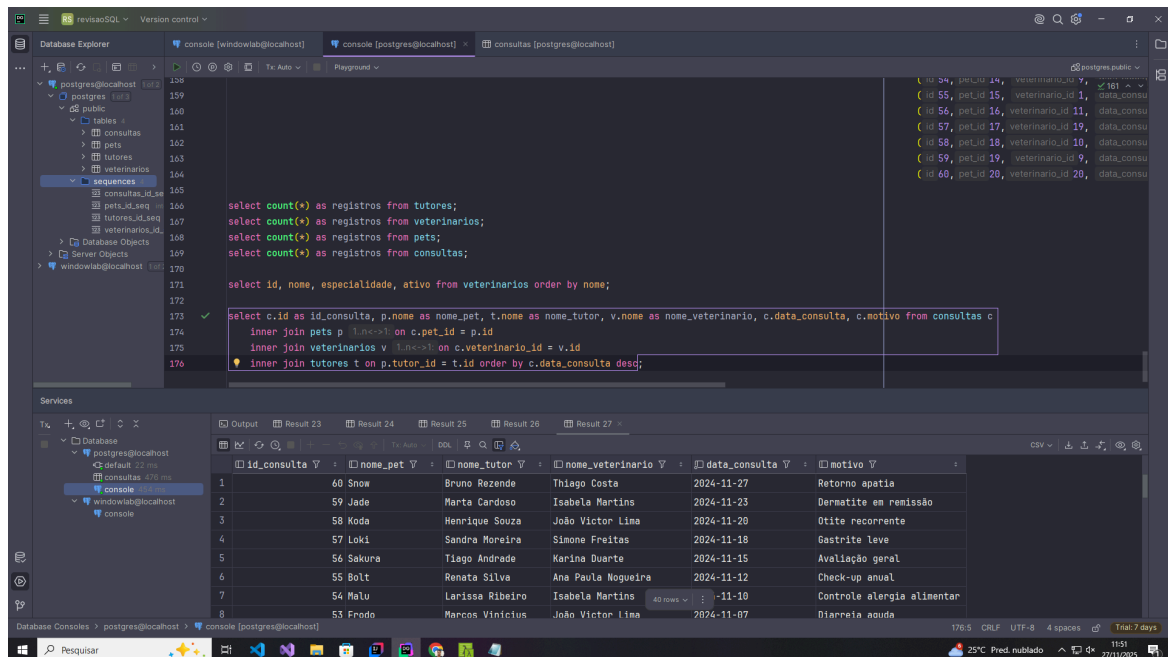
Questão 4 Consultas com JOIN entre tabelas

Liste todas as consultas realizadas, exibindo:

- consultas.id
- pets.nome como nome_pet
- tutores.nome como nome_tutor
- veterinarios.nome como nome_veterinario

- consultas.data_consulta
- consultas.motivo

Ordene da consulta mais recente para a mais antiga.

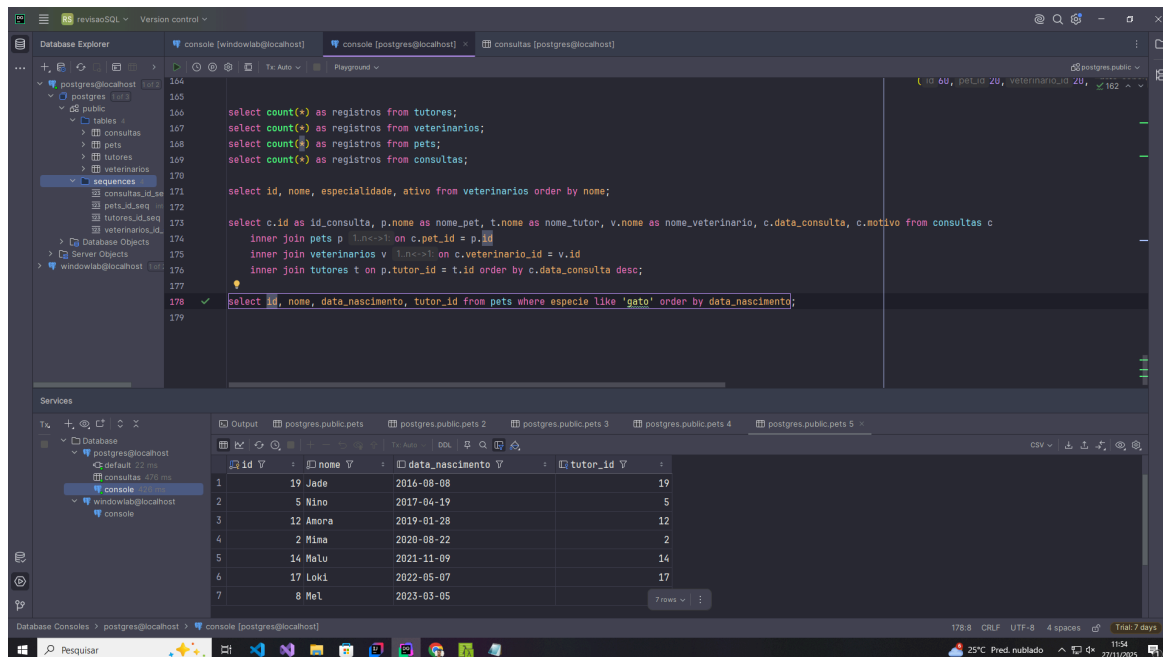


Questão 5 Pets de uma espécie específica

Liste todos os pets da espécie "gato", exibindo:

- id
- nome
- data_nascimento
- tutor_id

Ordene por data_nascimento do mais velho para o mais novo.

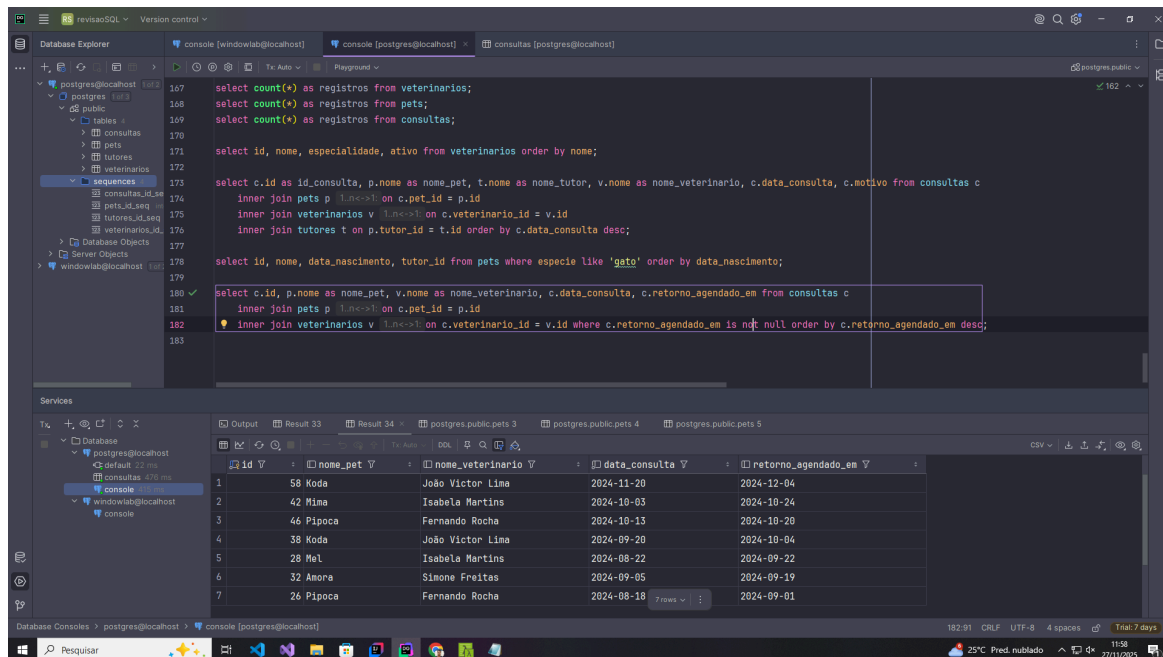


Questão 6 Retornos agendados

Liste todas as consultas que possuem um retorno_agendado_em preenchido (não nulo), exibindo:

- consultas.id
- pets.nome como nome_pet
- veterinarios.nome como nome_veterinario
- consultas.data_consulta
- consultas.retorno_agendado_em

Ordene pelo retorno_agendado_em do mais próximo para o mais distante.



Questão 7 Quantidade de consultas por veterinário

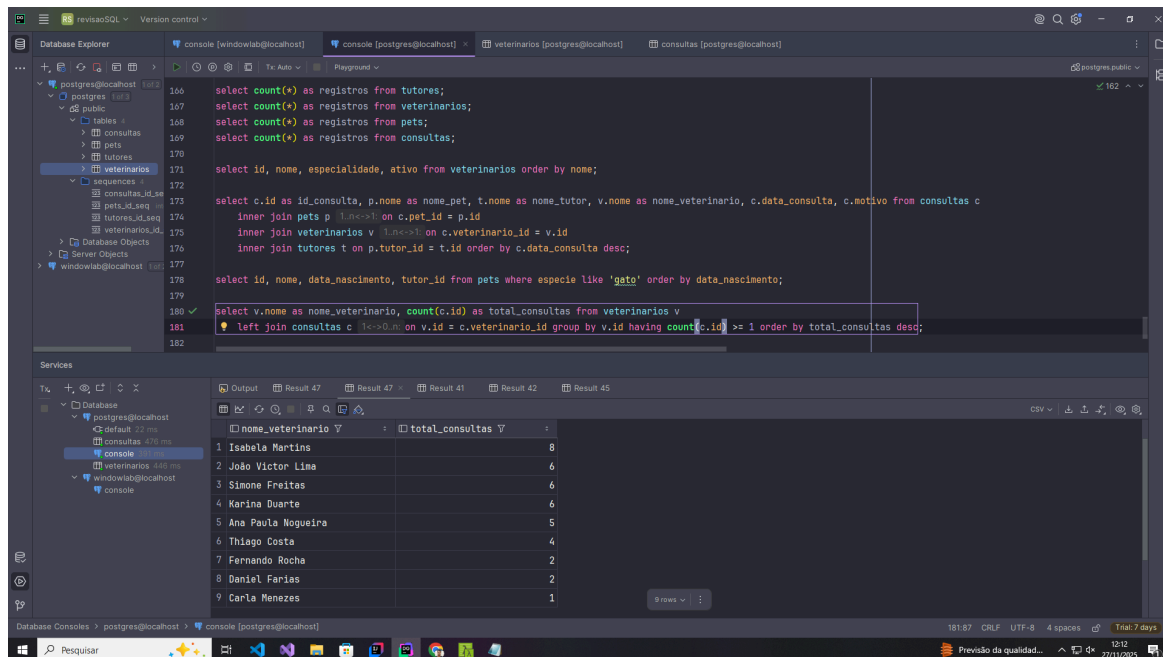
Mostre, para cada veterinário, quantas consultas ele já realizou.

A saída deve conter:

- nome_veterinario
- total_consultas

Mostre apenas veterinários que tenham pelo menos 1 consulta registrada.

Ordene do que mais atendeu para o que menos atendeu.



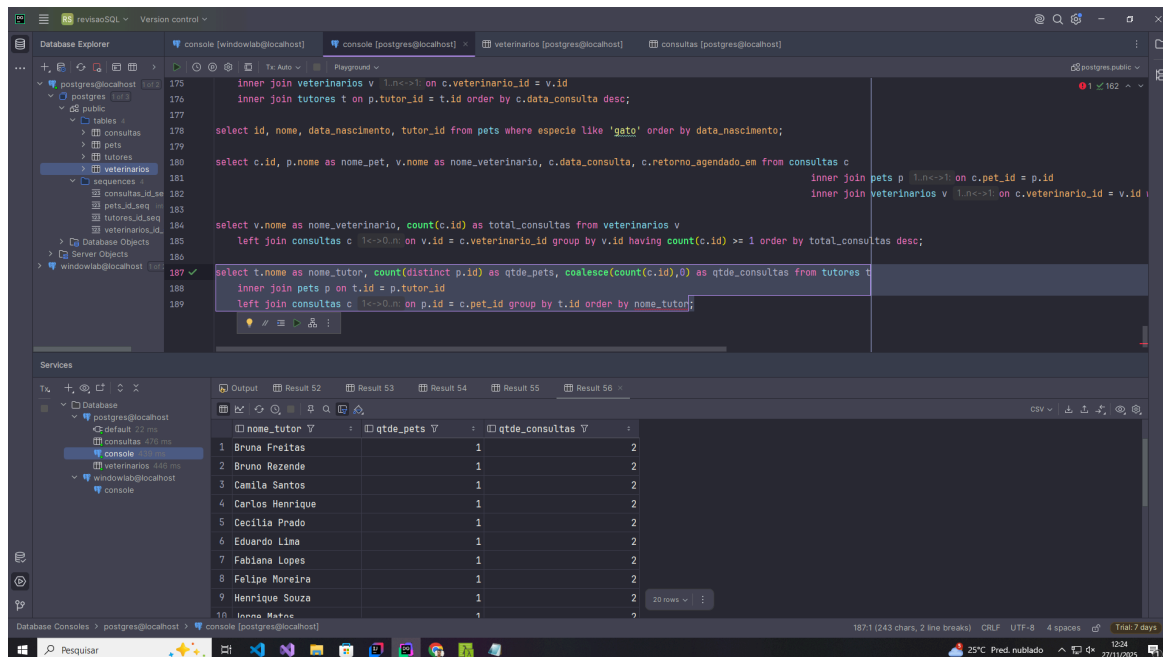
Questão 8 Tutores, pets e consultas

Monte uma query que mostre, para cada tutor:

- o nome_tutor
- o quantidade de pets que ele possui (qtde_pets)
- o quantidade total de consultas realizadas pelos seus pets (qtde_consultas)

Regras:

1. Tutores com pets, mas sem consultas, devem aparecer com zero consultas.
2. Ordene pelo nome_tutor.



Questão 9 Última consulta de cada pet

Escreva uma query que traga apenas a última consulta de cada pet, exibindo:

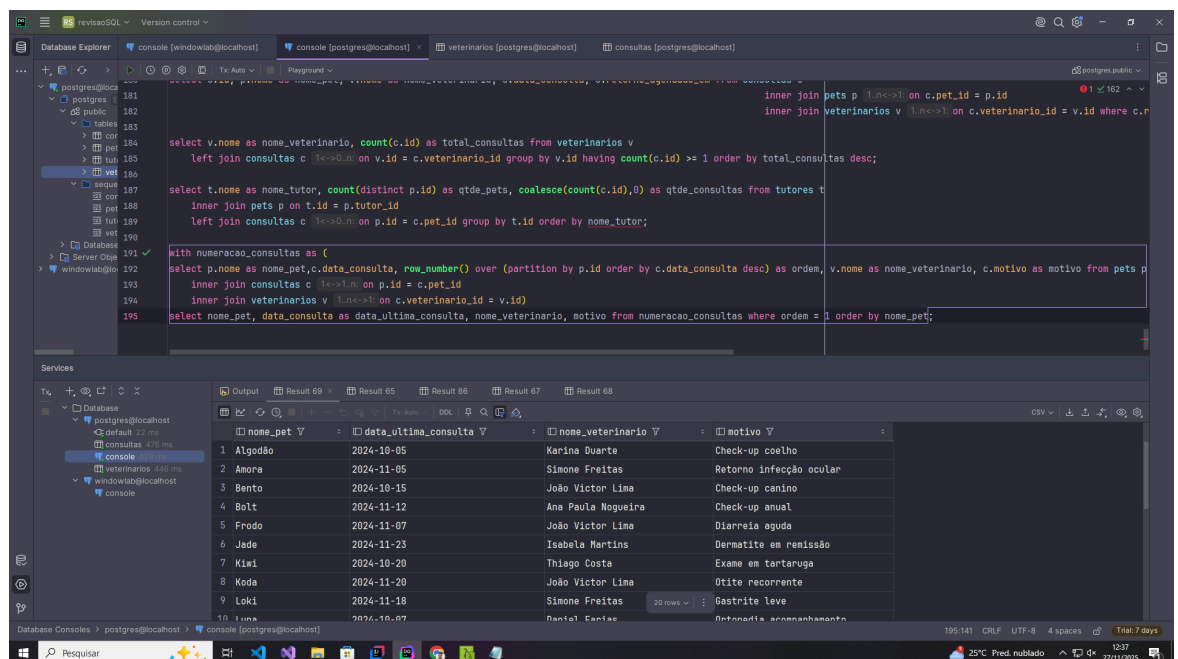
- nome_pet
- data_ultima_consulta
- nome_veterinario
- motivo

Regras:

1. Considere como “última” a consulta com a maior data_consulta de cada pet.
2. Use obrigatoriamente uma window function (ROW_NUMBER,

RANK ou similar) para identificar a última consulta dentro do conjunto de consultas de cada pet.

3. Depois, filtre para trazer somente a linha correspondente à última consulta de cada pet.
4. Ordene pelo nome_pet.



The screenshot shows a PostgreSQL IDE with a SQL editor and a results pane. The SQL editor contains the following queries:

```
select v.nome as nome_veterinario, count(c.id) as total_consultas from veterinarios v
left join consultas c on v.id = c.veterinario_id group by v.id having count(c.id) >= 1 order by total_consultas desc;

select t.nome as nome_tutor, count(distinct p.id) as qtde_pets, coalesce(count(c.id),0) as qtde_consultas from tutores t
inner join pets p on t.id = p.tutor_id
left join consultas c on p.id = c.pet_id group by t.id order by nome_tutor;

with numeracao_consultas as (
select p.nome as nome_pet, c.data_consulta, row_number() over (partition by p.id order by c.data_consulta desc) as orden, v.nome as nome_veterinario, c.motivo as motivo from pets p
inner join consultas c on p.id = c.pet_id
inner join veterinarios v on c.veterinario_id = v.id)
select nome_pet, data_consulta as data_ultima_consulta, nome_veterinario, motivo from numeracao_consultas where orden = 1 order by nome_pet;
```

The results pane shows the output of the last query, displaying 10 rows of data:

nome_pet	data_ultima_consulta	nome_veterinario	motivo
Algodão	2024-10-05	Karina Duarte	Check-up coelho
Amora	2024-11-05	Simone Freitas	Retorno infecção ocular
Bento	2024-10-15	João Victor Lima	Check-up canino
Bolt	2024-11-12	Ana Paula Nogueira	Check-up anual
Frodo	2024-11-07	João Victor Lima	Diarreia aguda
Jade	2024-11-23	Isabela Martins	Dermatite em remissão
Kiwi	2024-10-20	Thiago Costa	Exame em tartaruga
Koda	2024-11-20	João Victor Lima	Otitite recorrente
Loki	2024-11-18	Simone Freitas	Gastrite leve
Luna	2024-10-07	Daniel Cantas	Antenodia acompanhamento

Questão 10 Pets com múltiplas consultas

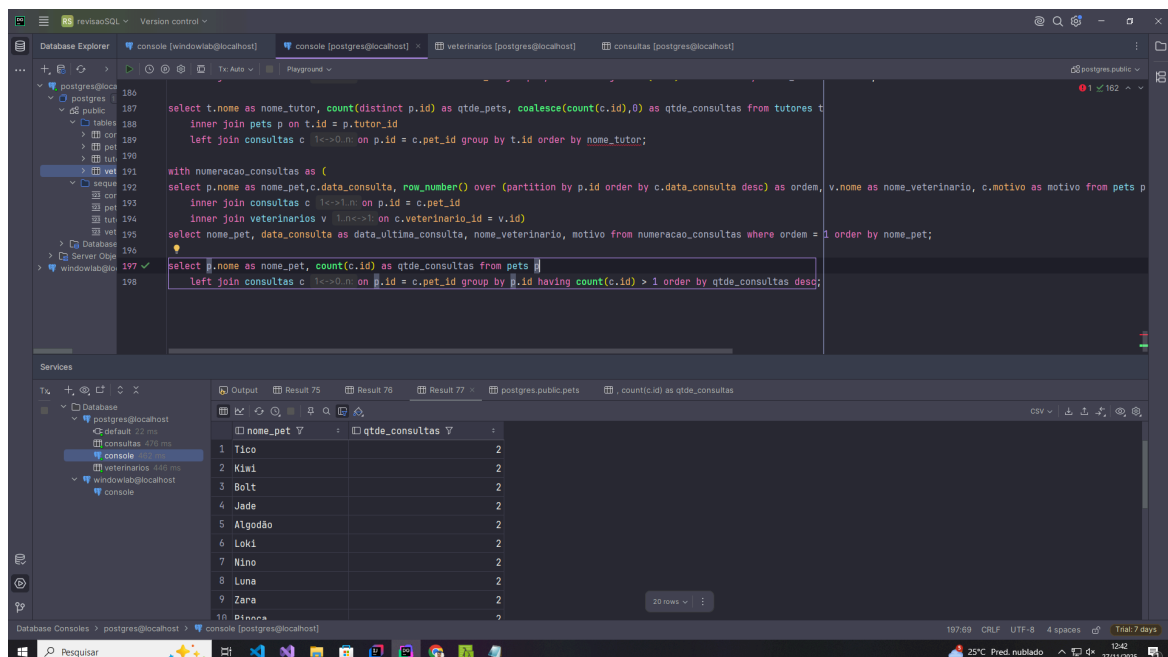
Liste todos os pets que já passaram por mais de 1 consulta na clínica.

A saída deve conter:

- nome_pet
- qtde_consultas

Mostre apenas os pets com qtde_consultas > 1, ordenando do que mais

consultou para o que menos.

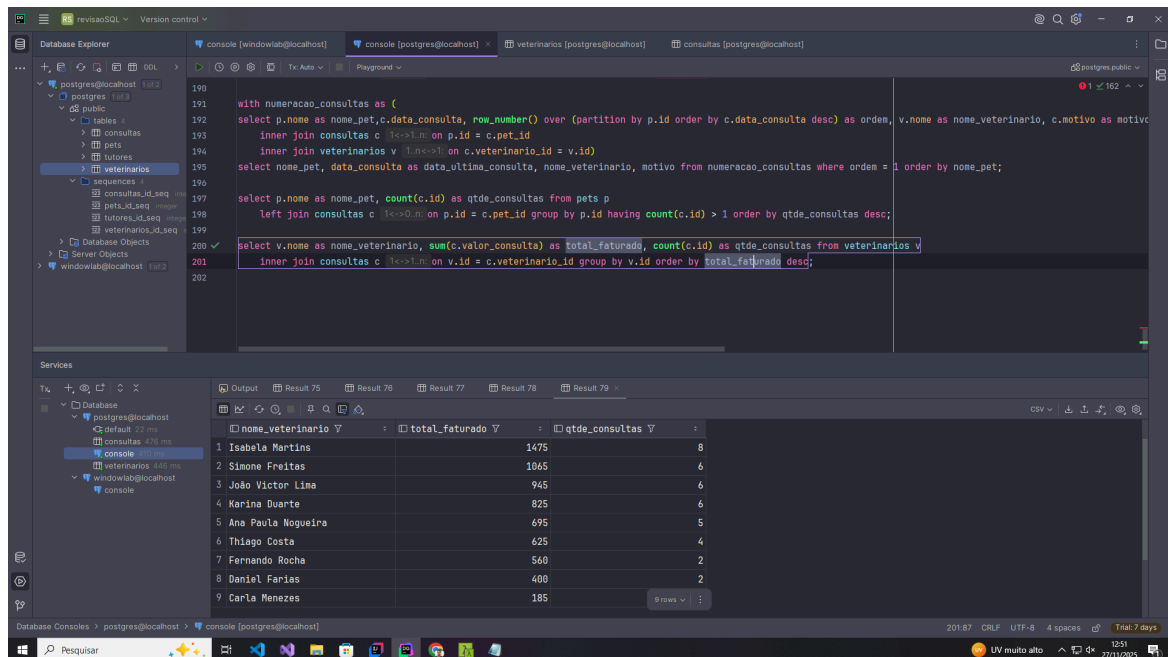


Questão 11 Faturamento por veterinário

Monte uma query que mostre, para cada veterinário:

- nome_veterinario
- total_faturado → soma de valor_consulta em todas as consultas realizadas por ele
- qtde_consultas → quantidade de consultas feitas

Considere todas as linhas da tabela consultas. Ordene o resultado do maior total_faturado para o menor.

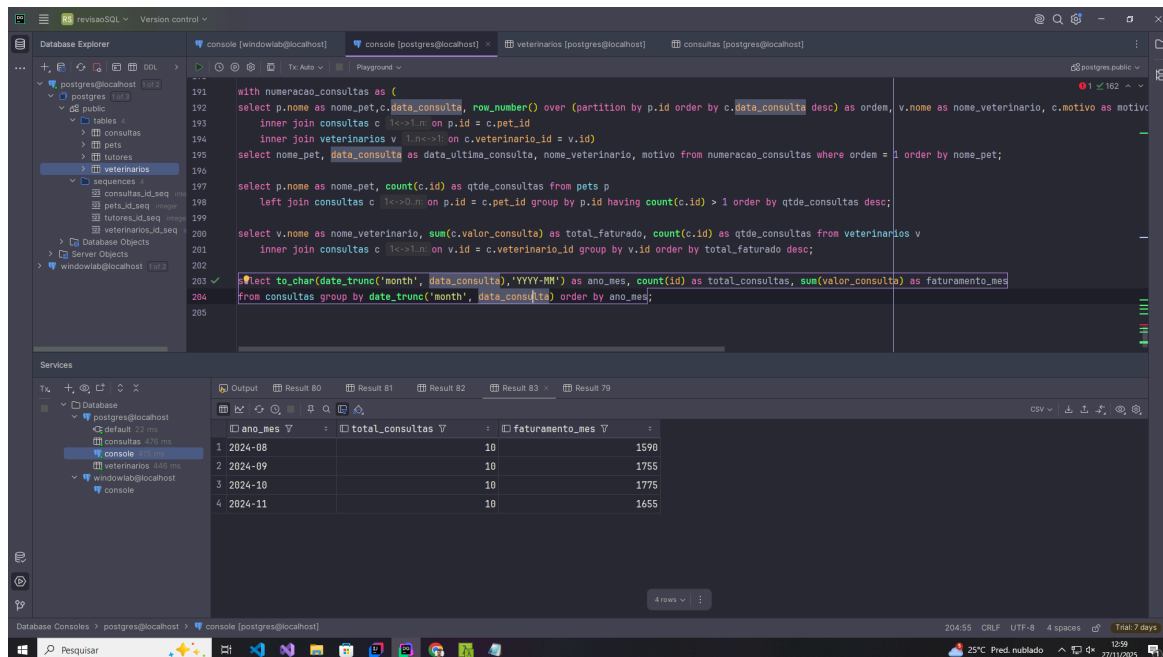


Questão 12 Faturamento mensal da clínica

Escreva uma query que calcule o faturamento mensal da clínica, exibindo:

- ano_mes (por exemplo, 2024-01, 2024-02 etc.)
- total_consultas → quantidade de consultas no mês
- faturamento_mes → soma de valor_consulta no mês

Você pode usar DATE_TRUNC ou funções equivalentes para agrupar por mês. Ordene por ano_mes do mais antigo para o mais recente.



Questão 13 Veterinário com maior faturamento em cada mês

Queremos descobrir, para cada mês, qual foi o veterinário com maior faturamento naquele mês.

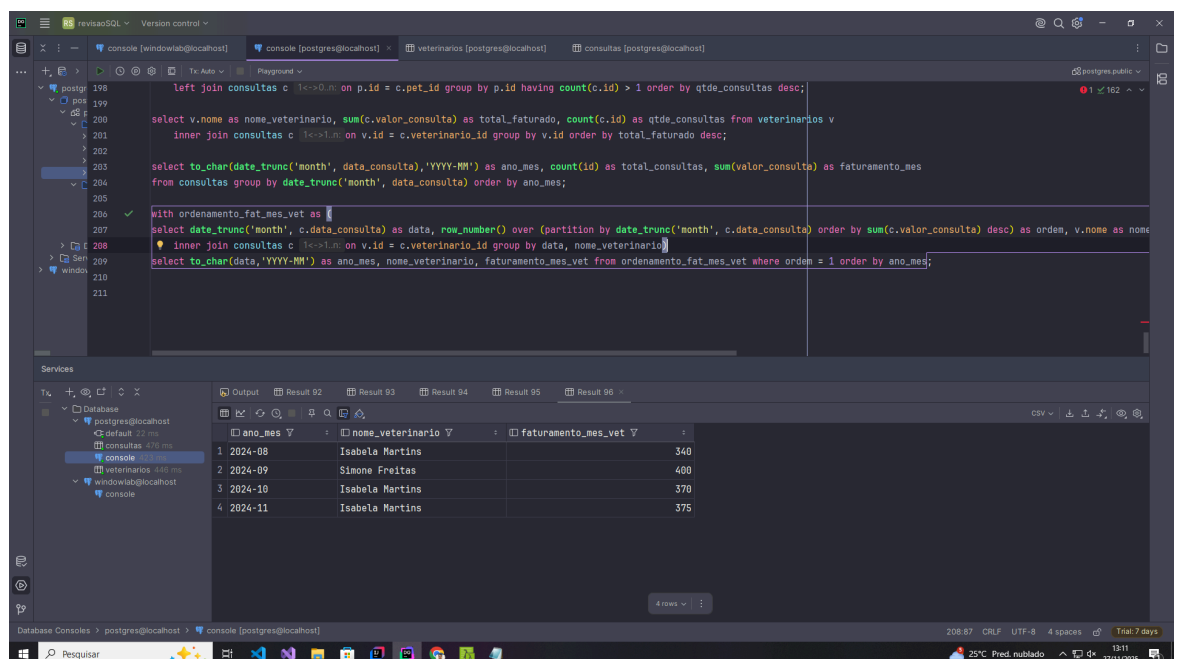
Monte uma query que produza:

- ano_mes
- nome_veterinario
- faturamento_mes_vet → soma de valor_consulta daquele veterinário naquele mês

Regras:

1. Primeiro, calcule o faturamento por combinação mês + veterinário.

2. Use uma window function (por exemplo, ROW_NUMBER ou RANK) particionada por mês para identificar o veterinário com maior faturamento em cada mês.
3. Retorne apenas o(s) veterinário(s) em 1º lugar em cada mês (em caso de empate, podem aparecer mais de um).
4. Ordene por ano_mes.



```
left join consultas c on p.id = c.pet_id group by p.id having count(c.id) > 1 order by qtde_consultas desc;

select v.nome as nome_veterinario, sum(c.valor_consulta) as total_faturado, count(c.id) as qtde_consultas from veterinarios v
inner join consultas c on v.id = c.veterinario_id group by v.id order by total_faturado desc;

select to_char(date_trunc('month', data_consulta), 'YYYY-MM') as ano_mes, count(id) as total_consultas, sum(valor_consulta) as faturamento_mes
from consultas group by date_trunc('month', data_consulta) order by ano_mes;

with ordenamento_fat_mes_vet as
select date_trunc('month', c.data_consulta) as data, row_number() over (partition by date_trunc('month', c.data_consulta) order by sum(c.valor_consulta) desc) as orden, v.nome as nome
inner join consultas c on v.id = c.veterinario_id group by data, nome_veterinario;
select to_char(data, 'YYYY-MM') as ano_mes, nome_veterinario, faturamento_mes_vet from ordenamento_fat_mes_vet where orden = 1 order by ano_mes;
```

ano_mes	nome_veterinario	faturamento_mes_vet
2024-08	Isabela Martins	340
2024-09	Simone Freitas	400
2024-10	Isabela Martins	370
2024-11	Isabela Martins	375

Questão 14 Participação de cada tutor no faturamento total

Queremos saber quanto cada tutor representa do faturamento total da clínica, considerando todas as consultas de todos os pets.

Monte uma query que produza:

- nome_tutor

- qtde_consultas → quantidade total de consultas feitas pelos pets daquele tutor
- total_gasto → soma de valor_consulta das consultas desses pets
- percentual_clinica → percentual que esse total_gasto representa em relação ao faturamento total da clínica

Regras:

1. Agrupe por tutor, usando tutores, pets e consultas.
2. Use uma window function para obter o faturamento total da clínica (soma de todas as valor_consulta).
3. Calcule percentual_clinica como $\text{total_gasto} / \text{faturamento_total} * 100$.
4. Ordene do tutor que mais gasta para o que menos gasta.

The screenshot shows a PostgreSQL database interface with a SQL query and its results. The query is as follows:

```

with ordenamento_fat_mes_vet as (
  select date_trunc('month', c.data_consulta) as data, row_number() over (partition by date_trunc('month', c.data_consulta) order by sum(c.valor_consulta) desc) as orden, v.nome
  from consultas c
  inner join veterinarios v on v.id = c.veterinario_id group by data, nome_veterinario)
select to_char(data, 'YYYY-MM') as ano_mes, nome_veterinario, faturamento_mes_vet from ordenamento_fat_mes_vet where orden = 1 order by ano_mes;

with tutor_consulta_info as (
  select t.nome as nome_tutor, count(c.id) as qtde_consultas, sum(c.valor_consulta) as total_gasto from tutores t
  inner join pets p on t.id = p.tutor_id
  left join consultas c on p.id = c.pet_id group by t.id)
select nome_tutor, qtde_consultas, total_gasto, round(100 * total_gasto / sum(total_gasto) over (), 2) as percentual_clinica from tutor_consulta_info order by total_gasto desc;
  
```

The results are displayed in a table with the following columns: nome_tutor, qtde_consultas, total_gasto, and percentual_clinica. The table shows 12 rows of data, ordered by total_gasto in descending order.

nome_tutor	qtde_consultas	total_gasto	percentual_clinica
Fabiana Lopes	2	560	8.27
Cecilia Prado	2	400	5.9
Larissa Ribeiro	2	390	5.76
Sandra Moreira	2	390	5.76
Camilla Santos	2	375	5.54
Marta Cardoso	2	375	5.54
Paula Siqueira	2	365	5.39
Marcos Vinicius	2	350	5.17
Jorge Matos	2	345	5.09
Bruno Rezende	2	335	4.94
Henrique Souza	2	320	4.72
Tiago Andrade	2	32	4.72

print com comentario do meu nome:

The screenshot shows a PostgreSQL console window with the following SQL queries and results:

```
--Richard
select count(*) as registros from tutores;
select count(*) as registros from veterinarios;
select count(*) as registros from pets;
select count(*) as registros from consultas;

--Richard
select id, nome, especialidade, ativo from veterinarios order by nome;

--Richard
select c.id as id_consulta, p.nome as nome_pet, t.nome as nome_tutor, v.nome as nome_veterinario, c.data_consulta, c.motivo from consultas c
inner join pets p on c.pet_id = p.id
inner join veterinarios v on c.veterinario_id = v.id
inner join tutores t on p.tutor_id = t.id order by c.data_consulta desc;

--Richard
select id, nome, data_nascimento, tutor_id from pets where especie like 'gato' order by data_nascimento;

--Richard
select c.id, p.nome as nome_pet, v.nome as nome_veterinario, c.data_consulta, c.retorno_agendado_em from consultas c
inner join pets p on c.pet_id = p.id
inner join veterinarios v on c.veterinario_id = v.id when
```

The results pane shows a table with 3 rows and 4 columns:

nome_tutor	qtde_consultas	total_gasto	percentual_clinica
Fabiana Lopes	2	560	8.27
Cecilia Prado	2	400	5.9
Larissa Ribeiro	2	39	5.76

The screenshot shows a PostgreSQL console window with the following SQL queries and results:

```
--Richard
select v.nome as nome_veterinario, count(c.id) as total_consultas from veterinarios v
left join consultas c on v.id = c.veterinario_id group by v.id having count(c.id) >= 1 order by total_consultas desc;

--Richard
select t.nome as nome_tutor, count(distinct p.id) as qtde_pets, coalesce(count(c.id),0) as qtde_consultas from tutores t
inner join pets p on t.id = p.tutor_id
left join consultas c on p.id = c.pet_id group by t.id order by nome_tutor;

--Richard
with numeracao_consultas as (
select p.nome as nome_pet, c.data_consulta, row_number() over (partition by p.id order by c.data_consulta desc) as orden, v.nome as nome_veterinario, c.motivo as motivo from pets p
inner join consultas c on p.id = c.pet_id
inner join veterinarios v on c.veterinario_id = v.id)
select nome_pet, data_consulta as data_ultima_consulta, nome_veterinario, motivo from numeracao_consultas where orden = 1 order by nome_pet;

--Richard
select p.nome as nome_pet, count(c.id) as qtde_consultas from pets p
left join consultas c on p.id = c.pet_id group by p.id having count(c.id) > 1 order by qtde_consultas desc;

--Richard
select v.nome as nome_veterinario, sum(c.valor_consulta) as total_faturado, count(c.id) as qtde_consultas from veterinarios v
inner join consultas c on v.id = c.veterinario_id group by v.id order by total_faturado desc;
```

The results pane shows a table with 3 rows and 4 columns:

nome_tutor	qtde_consultas	total_gasto	percentual_clinica
Fabiana Lopes	2	560	8.27
Cecilia Prado	2	400	5.9
Larissa Ribeiro	2	39	5.76

